



BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Semana 05



Nuevos paradigmas en el manejo de datos. Bases de datos NoSQL. Operaciones CRUD en NoSQL.
Business intelligence, Fases de un proyecto BI.



Presentación de la sesión

Logro de sesión

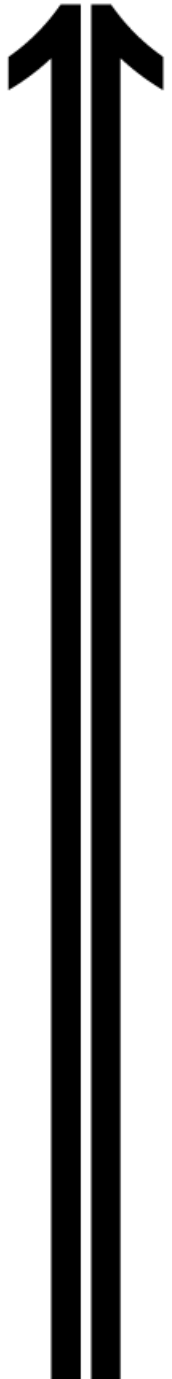
Al final de la sesión el alumno podrá interactuar con una base de datos NoSQL , también comprenderá como se ejecutan las operaciones CRUD, además conocerá los conceptos de Business intelligence, su importancia y las fases para la ejecución de un proyectos de Inteligencia de Negocios.

Temario

- Nuevos paradigmas en el manejo de datos. Bases de datos NoSQL.
- Operaciones CRUD en NoSQL.
- Business intelligence, Fases de un proyecto BI



¿Qué es una base de datos noSQL?



NOSQL



¿Qué es una base de datos NoSQL?

NoSQL = **N**ot **O**nly **SQL***



*No solo SQL



¿Qué es una base de datos NoSQL?

NoSQL se refiere a una gran variedad de tecnologías de bases de datos que se han desarrollado en respuesta a las necesidades de desarrollo de las aplicaciones modernas:

- Los desarrolladores trabajan con aplicaciones que generan enormes volúmenes de datos nuevos y en constante evolución (estructurados, semiestructurados, no estructurados y polimórficos).
- Aquellos ciclos de desarrollo en cascada que duraban entre 12 y 18 meses hace tiempo que pasaron a la historia. Ahora se trabaja en equipos pequeños, que realizan sprints de desarrollo ágiles con iteraciones rápidas, y que generan código cada semana o cada quince días, algunos incluso varias veces al día.
- Las aplicaciones que antes servían a un conjunto finito de usuarios ahora se proporcionan como servicios, que no solo deben funcionar sin interrupción, sino que además tienen que ser accesibles desde muchos dispositivos distintos y deben poder escalarse a millones de usuarios de todo el mundo.



¿Qué es una base de datos NoSQL?

- Las organizaciones ahora están recurriendo a arquitecturas de escalamiento horizontal que utilizan tecnologías de software abierto, servidores básicos y computación en la nube en lugar de grandes servidores monolíticos e infraestructura de almacenamiento.

Las bases de datos relacionales no se diseñaron para poder hacer frente a la escalabilidad y agilidad que necesitan las aplicaciones modernas, ni para beneficiarse de los sistemas de almacenamiento básicos y de la potencia de proceso que existen hoy en día.

Las bases de datos NoSQL son bases de datos diseñadas para usarse en grandes sistemas de desconfianza. Son notablemente mucho más escalables y mucho más rápidos en el manejo de cargas de datos muy grandes que las bases de datos relacionales tradicionales. A diferencia de otras bases de datos, las bases de datos NoSQL no utilizan las relaciones tabulares estándar de las bases de datos relacionales.



Algunas BD NoSQL

- RavenDB
- CouchBase
- Redis
- MongoDB
- IBMCloudant
- Amazon DynamoDB
- Cassandra
- Azure CosmosDB
- ArangoDB
- Google Cloud Datastore
- Apache Drill
- Amazon DocumentDB
- Oracle NoSQL Database
- Amazon Neptune
- TeraText Database System
- Qubole
- Aerospike Database
- DataStax
- Riak
- MarkLogic Server
- Hbase
- Percona Server
- Scylla
- Voldemort



NoSQL Funciones y Capacidades

- Rendimiento
- Disponibilidad
- Multimodelo
- Concurrencia
- Seguridad
- Escalabilidad
- Flexibilidad del modelo de datos
- Flexibilidad del modelo de implementación



¿Ventajas de NoSQL sobre las bases de datos relacionales?

- Las bases de datos relacionales tradicionales almacenan datos en forma tabular altamente estructurada, con múltiples filas y columnas. Si bien estos almacenes de datos son altamente flexibles, fáciles de mantener y efectivos para los datos almacenados en un solo servidor, no escalan muy bien en un sistema distribuido en comparación con las alternativas de bases de datos NoSQL.
- Los sistemas distribuidos que utilizan una capacidad de procesamiento y almacenamiento de bajo costo son cada vez más comunes y, a menudo, se utilizan en entornos donde existe una necesidad de alta disponibilidad y velocidad. Las bases de datos NoSQL funcionan significativamente mejor en este tipo de sistema distribuido.



¿Ventajas de NoSQL sobre las bases de datos relacionales?

Las otras ventajas principales son:

- Las bases de datos NoSQL son no relacionales, lo que significa que no tienen tablas y son más fáciles de administrar. Proporcionan un mayor nivel de flexibilidad con modelos de datos más nuevos.
- A menudo son de código abierto y, por lo tanto, de menor costo. Las bases de datos NoSQL pueden ser una solución atractiva para organizaciones más pequeñas con presupuestos limitados.
- Las bases de datos NoSQL están optimizadas para modelos de datos específicos que permiten un mayor rendimiento que intentar lograr una funcionalidad similar con bases de datos relacionales.



Modelo de Datos NoSQL

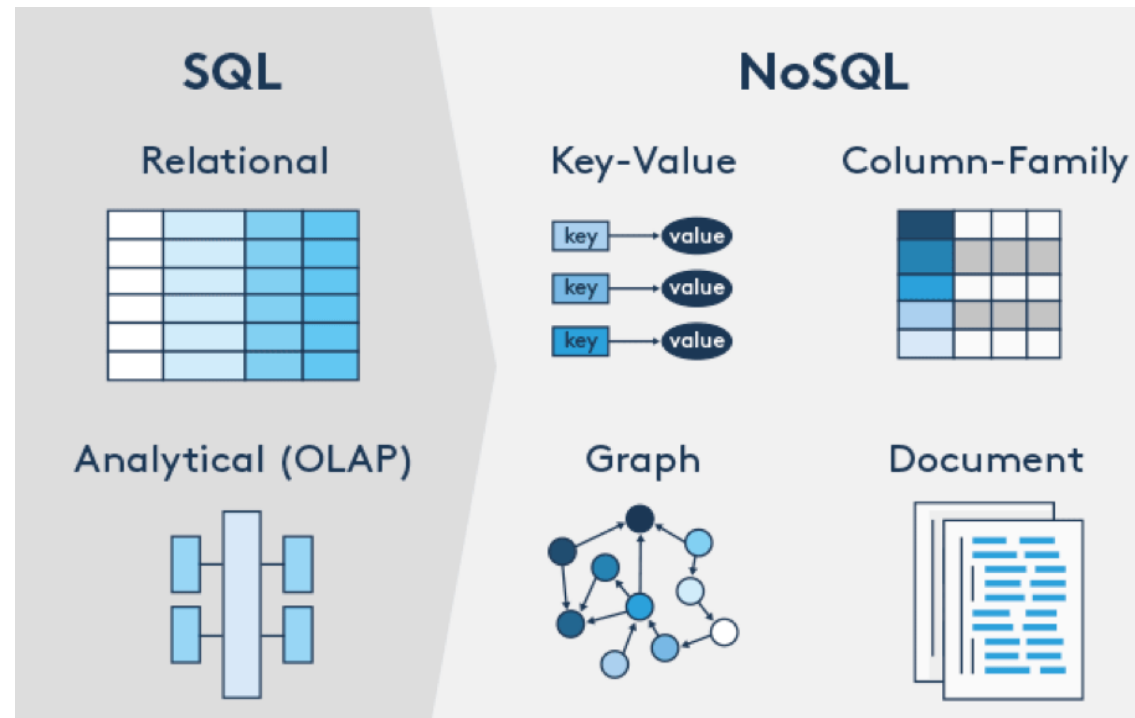
No todas las bases de datos NoSQL utilizan el mismo modelo de datos. Aunque la distinción principal entre un almacén de datos relacional y una base de datos NoSQL es el modelo de datos, existen varios tipos diferentes de modelo de datos NoSQL. A continuación se muestran descripciones de tres de los modelos de datos más populares utilizados en las bases de datos NoSQL:

- **Modelo de documento:** estas bases de datos NoSQL reemplazan la estructura familiar de filas y columnas con un modelo de almacenamiento de documentos. Cada documento está estructurado, con frecuencia utilizando el modelo JavaScript Object Notation (JSON). El modelo de datos del documento está asociado con la programación orientada a objetos donde cada documento es un objeto.
- **Modelo gráfico:** las bases de datos NoSQL que utilizan el modelo gráfico generalmente requieren que todos los datos residan en una máquina, lo que niega una de las ventajas clave de las bases de datos NoSQL. Esta clase de bases de datos utiliza estructuras como nodos de datos, bordes y propiedades, lo que facilita el modelado de relaciones entre entidades en una aplicación.



Modelo de Datos NoSQL

- Modelo de clave-valor: en este modelo de base de datos NoSQL, se requiere una clave para recuperar y actualizar datos. El modelo de datos clave-valor es muy simple y, por lo tanto, se escala bien. Sin embargo, esta simplicidad y escalabilidad se obtienen a costa de la complejidad de las consultas.





MONGODB



¿Qué es MongoDB y cómo funciona?

- MongoDB es una base de datos NoSQL orientada a documentos.



¿Cómo funciona?

MongoDB	SQL
Base de Datos	Base de Datos
Colecciones	Tablas
Documentos	Filas (registros)
Campos	Columnas



Ejemplo

¿Cómo funciona?

Base de Datos	Tienda	
Colecciones	Usuarios	Pedidos
Documentos	{Nombre: 'Juan', edad: 25}	{...}
	{Nombre: 'Felipe'}	{...}



Modelo de Datos NoSQL

- Colección: un almacén organizado de documentos en MongoDB generalmente con campos comunes entre documentos. Puede haber muchas colecciones por base de datos y muchos documentos por colección.
- Documento: una forma de organizar y almacenar datos como un conjunto de pares de valor de campo. Los documentos son esquimales, es decir, que no tienen esquema.
- Campo: un identificador único para un punto de datos. Puedes tener 1 campo, 2 campos, no importa la cantidad. Esto nos da flexibilidad en las aplicaciones porque no se tienen que crear tablas con valores vacíos, se crean los documentos y si tienen valor se utiliza o no.
- Valor: datos relacionados con un identificador determinado.



Formato de Uso

- Los documentos se guardan en Formato JSON (BSON viene de Binary).
- Empiezan con llaves y son de clave valor.

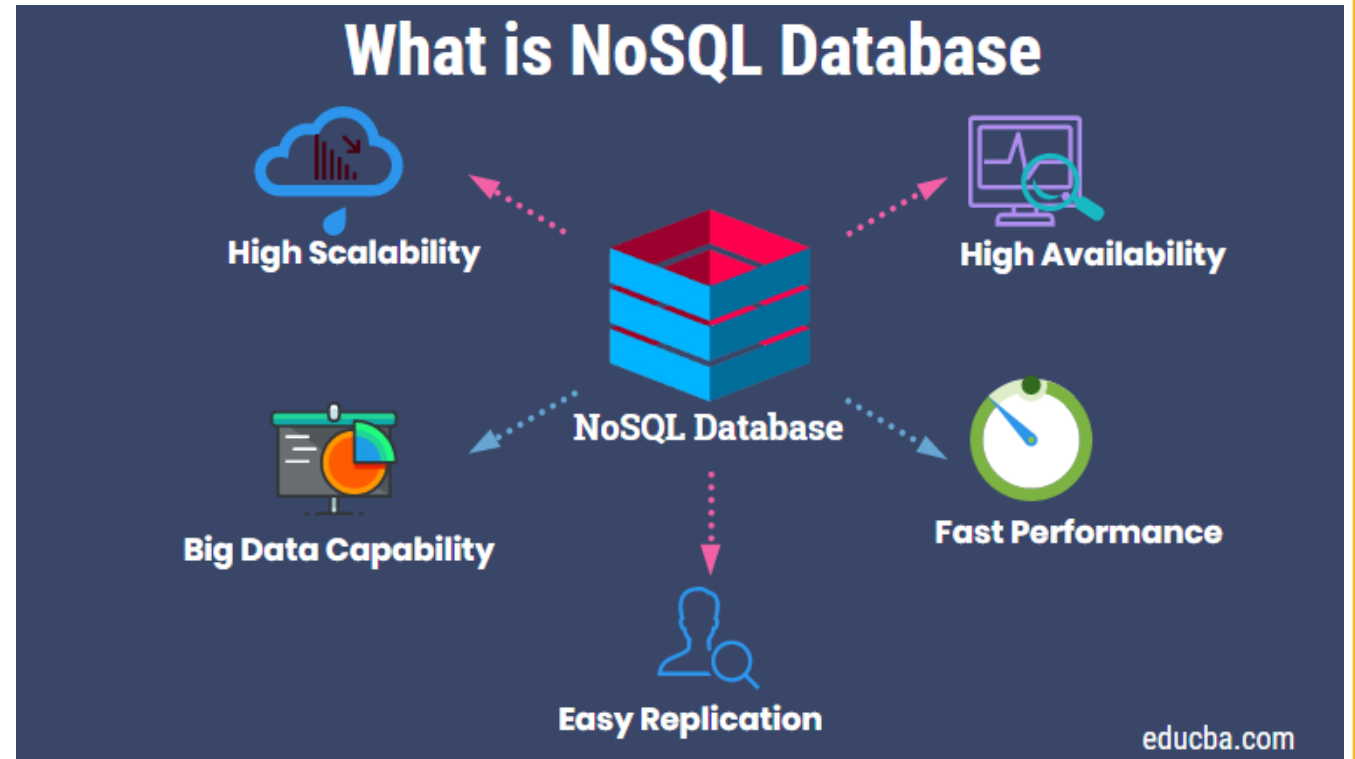
JSON (BSON)

```
{
  "nombre": "Juan",
  "edad": 25
  "dirección":
    {
      "ciudad": "Barcelona"
    },
  "aficiones":[
    {"nombre": "Fútbol" },
    {"nombre": "Esquí" }
  ]
}
```



Ventajas de MongoDB

- Más rápido ya que almacena todo en un mismo sitio (una colección).
- No hace falta unir tablas como en SQL.
- Más flexibilidad para las aplicaciones y crear relaciones complejas.





Nube

[Pruebe gratis MongoDB Atlas](#)

Entorno de prueba

- Gratis para siempre
- Ideal para aprender, desarrollar y crear prototipos

Compartida

- Hasta 5 GB de almacenamiento
- RAM compartida

Dedicada

- Rendimiento coherente
- Seguridad avanzada
- Escalamiento ilimitado

Servidor

[Descargar MongoDB](#)

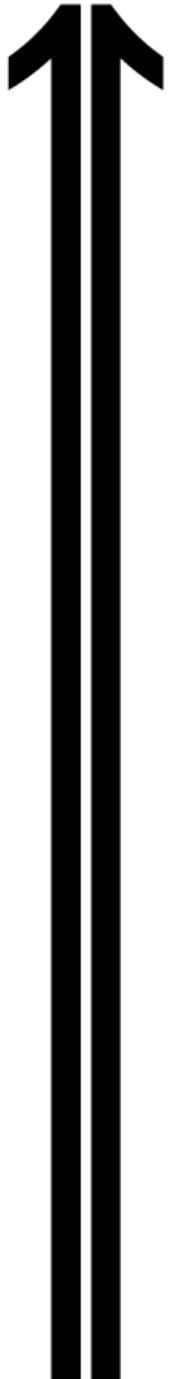
MongoDB ofrece dos versiones de su potente base de datos de documentos distribuida: Enterprise y Community.

Comunidad

- Múltiples funciones
- Listo para los desarrolladores

Empresa

- Funciones avanzadas
- Alto rendimiento



OPERACIONES CRUD

OPERACIONES CRUD



Definición

- Las operaciones CRUD vienen de las siglas siguientes:
 - CREATE → CREAR
 - READ → LEER
 - UPDATE → ACTUALIZAR
 - DELETE → BORRAR
- Estas son todas las operaciones que podemos realizar en una base de datos.

OPERACIONES CRUD

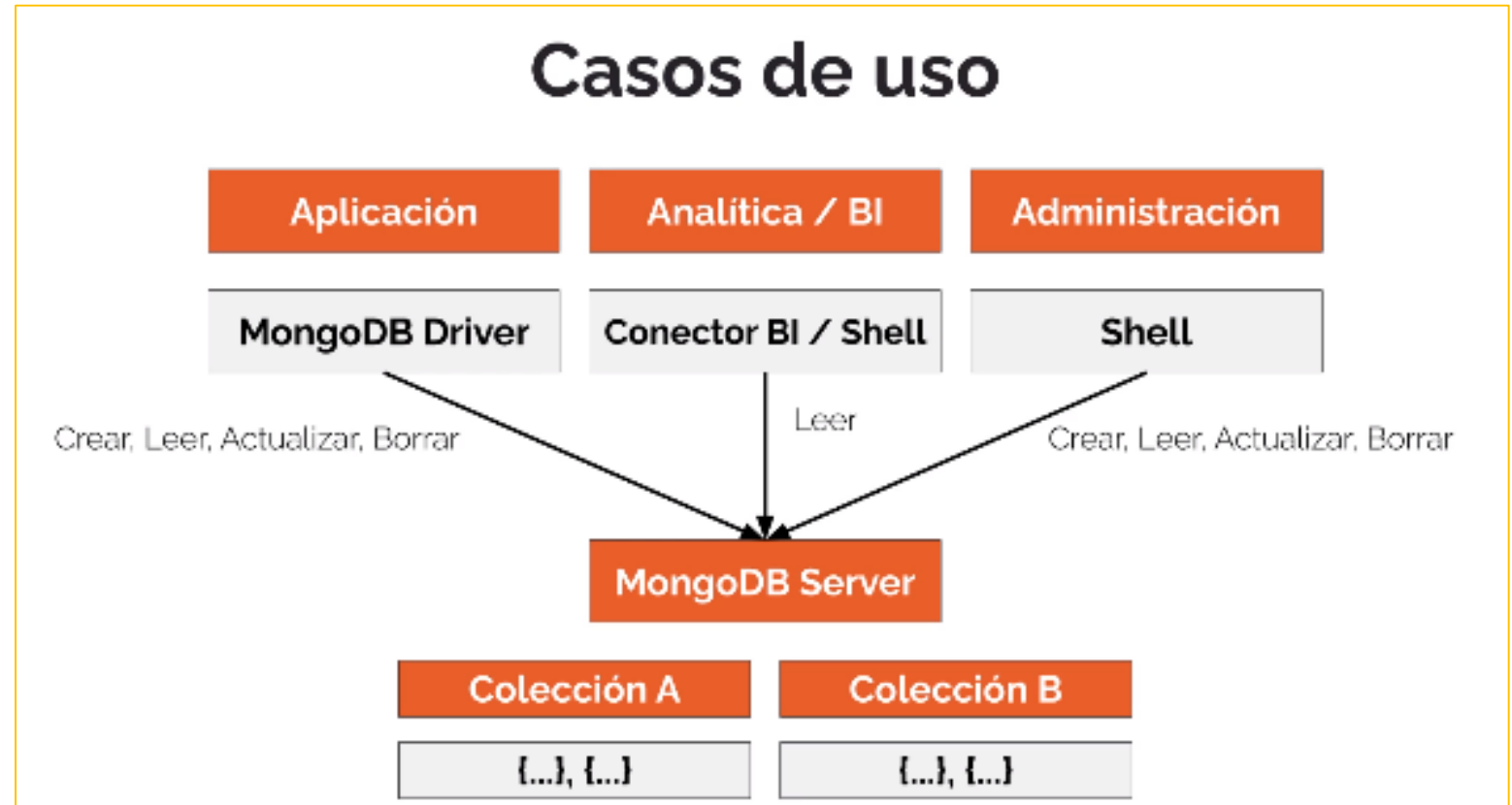


Casos de Uso

Están enfocados en las diversas operaciones que se puede realizar en una base de datos MongoDB.

Esas operaciones pueden ser de :

- Aplicaciones
- Analítica/BI
- Administración de BD

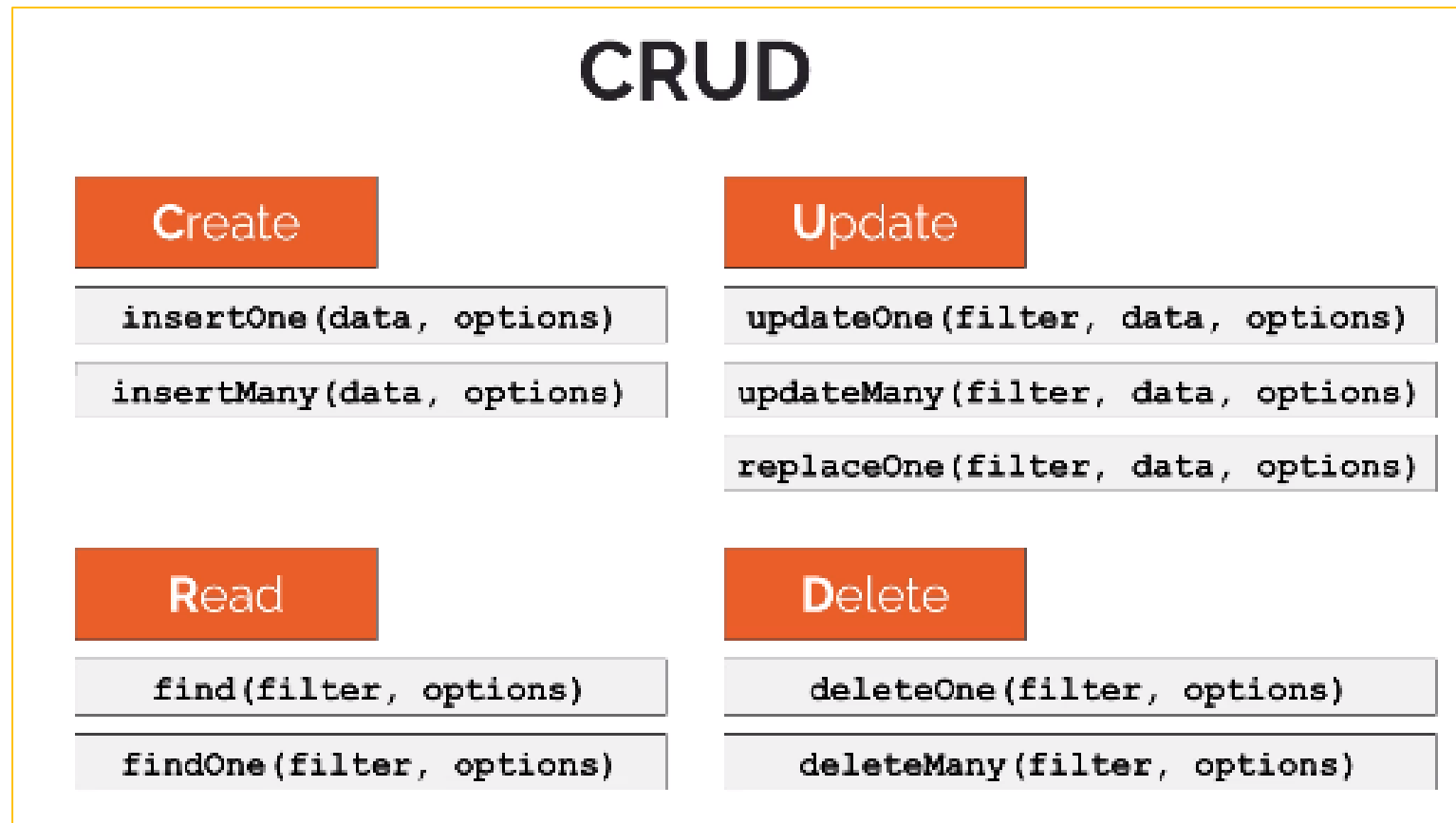


OPERACIONES CRUD



CRUD

A continuación se muestran algunas operaciones básicas que se pueden realizar dentro de una base de datos MongoDB.



OPERACIONES CRUD



1.CREATE (Crear)

- `db.collection.insertOne()` - Inserta un solo documento en la colección especificada.
- `db.collection.insertMany()` - Inserta varios documentos en la colección especificada.

2.READ (Leer)

- `db.collection.find()` - Devuelve todos los documentos de la colección especificada que coincidan con el criterio de búsqueda.
- `db.collection.findOne()` - Devuelve el primer documento que coincida con el criterio de búsqueda.
- `db.collection.count()` - Devuelve el número de documentos que coinciden con el criterio de búsqueda.

3.UPDATE (Actualizar)

- `db.collection.updateOne()` - Actualiza un solo documento que coincida con el criterio de búsqueda.
- `db.collection.updateMany()` - Actualiza varios documentos que coincidan con el criterio de búsqueda.
- `db.collection.replaceOne()` - Reemplaza un solo documento que coincida con el criterio de búsqueda.

4.DELETE (Eliminar)

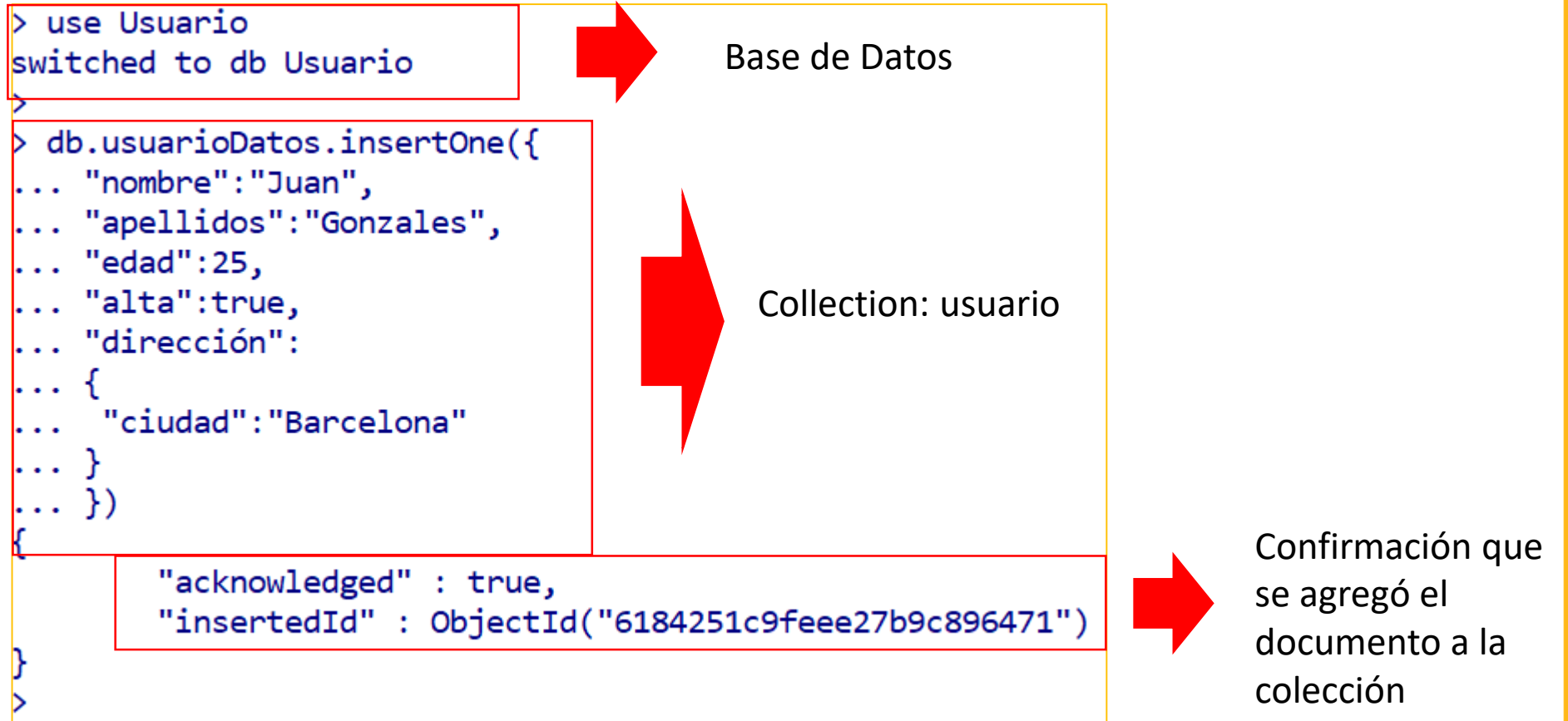
- `db.collection.deleteOne()` - Elimina un solo documento que coincida con el criterio de búsqueda.
- `db.collection.deleteMany()` - Elimina varios documentos que coincidan con el criterio de búsqueda.

OPERACIONES CRUD



Ejemplo:

Creamos una BD Usuario con el comando **use** y le agregamos un conjunto de colecciones



OPERACIONES CRUD



Ejemplo:

Ingresamos más colecciones utilizando el comando **insertOne()**

```
> db.ejemploDatos.insertOne( {      "nombre": "Felipe",      "apellidos": "Sanchez",      "edad": 27,
  "alta": false,      "dirección":      {      "ciudad": "Madrid"      }      })
{
  "acknowledged" : true,
  "insertedId" : ObjectId("61843ae19feee27b9c896472")
}
```

En esta colección ingresamos manualmente el ID de cada documento:

```
> db.usuariosDatos.insertOne({nombre:"Pablo", _id:"123"})
{ "acknowledged" : true, "insertedId" : "123" }
> db.usuariosDatos.insertOne({nombre:"Alex", _id:"1234"})
{ "acknowledged" : true, "insertedId" : "1234" }
```

OPERACIONES CRUD



Ejemplo:

Podemos ingresar documentos con el mismo campo pero el ID tiene que ser diferente.

```
> db.usuariosDatos.insertOne({nombre:"Pablo Ramos", _id:"12345"})
{ "acknowledged" : true, "insertedId" : "12345" }
> db.usuariosDatos.insertOne({nombre:"Pablo Ramos"})
{
  "acknowledged" : true,
  "insertedId" : ObjectId("61843b8a17c96a3963a804c2")
}
```

OPERACIONES CRUD



Visualización de documentos con el comando **find()**

```
> db.usuariosDatos.find()
{ "_id" : "12345", "nombre" : "Pablo Ramos" }
{ "_id" : ObjectId("61843b8a17c96a3963a804c2"), "nombre" : "Pablo Ramos" }
{ "_id" : "1234", "nombre" : "Alex" }
{ "_id" : "123", "nombre" : "Pablo" }
>
```

O si deseamos verlo en una mejor presentación con el comando **find().pretty()**

```
> db.ejemploDatos.find().pretty()
{
  "_id" : ObjectId("61843ae19feee27b9c896472"),
  "nombre" : "Felipe",
  "apellidos" : "Sanchez",
  "edad" : 27,
  "alta" : false,
  "dirección" : {
    "ciudad" : "Madrid"
  }
}
```

OPERACIONES CRUD



Para eliminar objetos, seleccionamos por campo o valor y utilizamos el comando `deleteOne()`

```
> db.usuariosDatos.deleteOne({nombre:"Alex"})  
{ "acknowledged" : true, "deletedCount" : 1 }
```

OPERACIONES CRUD



También podemos actualizar un documento de la colección con un marcador (**\$set**) utilizando el comando **updateOne()**

```
> db.usuariosDatos.find().pretty()
{ "_id" : "123", "nombre" : "Pablo" }
{ "_id" : "1234", "nombre" : "Alex" }
{ "_id" : ObjectId("6184407017c96a3963a804c3"), "nombre" : "Pablo Ramos" }
{ "_id" : ObjectId("6184409a17c96a3963a804c4"), "nombre" : "Juan" }
>
>
>
>
> db.usuariosDatos.updateOne({nombre:"Juan"},{$set: {marcador:"borrar"}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
>
> db.usuariosDatos.find().pretty()
{ "_id" : "123", "nombre" : "Pablo" }
{ "_id" : "1234", "nombre" : "Alex" }
{ "_id" : ObjectId("6184407017c96a3963a804c3"), "nombre" : "Pablo Ramos" }
{
  "_id" : ObjectId("6184409a17c96a3963a804c4"),
  "nombre" : "Juan",
  "marcador" : "borrar"
}
```

OPERACIONES CRUD



O marcar todos los documentos de la colección con el comando **updateMany()**

```
> db.usuariosDatos.updateOne({},{$set: {marcador:"borrar"}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.usuariosDatos.updateMany({},{$set: {marcador:"borrar"}})
{ "acknowledged" : true, "matchedCount" : 4, "modifiedCount" : 2 }
>
> db.usuariosDatos.find().pretty()
{ "_id" : "123", "nombre" : "Pablo", "marcador" : "borrar" }
{ "_id" : "1234", "nombre" : "Alex", "marcador" : "borrar" }
{
  "_id" : ObjectId("6184407017c96a3963a804c3"),
  "nombre" : "Pablo Ramos",
  "marcador" : "borrar"
}
{
  "_id" : ObjectId("6184409a17c96a3963a804c4"),
  "nombre" : "Juan",
  "marcador" : "borrar"
}
>
```

OPERACIONES CRUD



Eliminar todos los registros con el comando `deleteMany()`:

```
> db.usuariosDatos.deleteMany({marcador:"borrar"})  
{ "acknowledged" : true, "deletedCount" : 4 }  
>  
> db.usuariosDatos.find().pretty()
```

OPERACIONES CRUD



Insertamos varios documentos con el comando **insertMany()**

```
> db.usuariosDatos.insertMany([ {  
...   "nombre": "Juan",  
...   "apellidos": "Gonzalez",  
...   "edad": 25,  
...   "alta": true,  
...   "dirección": {  
...     "ciudad": "Barcelona"  
...   }  
... }, {  
...   "nombre": "Felipe",  
...   "apellidos": "Sanchez",  
...   "edad": 27,  
...   "alta": false,  
...   "dirección": {  
...     "ciudad": "Madrid"  
...   } } ])  
{  
  "acknowledged" : true,  
  "insertedIds" : [  
    ObjectId("6184457117c96a3963a804c5"),  
    ObjectId("6184457117c96a3963a804c6")  
  ]  
}  
>
```

OPERACIONES CRUD



Ejemplo:

El comando **find()**

```
>db.usuarioDatos.find().pretty()
```

```
>db.usuarioDatos.find({nombre:"Juan"}).pretty()
```

```
>db.usuarioDatos.find({edad:25}).pretty()
```

```
>db.usuarioDatos.find({edad: {$gt:24} }).pretty()
```

```
>db.usuarioDatos.findOne({edad: {$gt:24} })
```

Greater Than = mayor que

FindOne devuelve sólo 1 registro y busca el primero ordenado por el objeto ID.

OPERACIONES CRUD



Ejemplo:

Comandos **updateOne()** y **replaceOne()**

```
> db.usuarioDatos.find().pretty() //para seleccionar un id
```

```
> db.usuarioDatos.updateOne({_id:ObjectId("618057676a7b9de8e9bce9ff")}, {$set: {alta:true}})
```

```
> db.usuarioDatos.update({_id:ObjectId("618057676a7b9de8e9bce9ff")}, {$set: {alta:false}})
```

```
> db.usuarioDatos.update({_id:ObjectId("618057676a7b9de8e9bce9ff")}, {alta:false})
```

```
> db.usuarioDatos.replaceOne({_id:ObjectId("618057676a7b9de8e9bce9ff")}, {      "nombre":  "Felipe",  
"apellidos": "Sanchez",  "edad": 27,  "alta": false,  "dirección": {    "ciudad": "Madrid"  })
```

OPERACIONES CRUD



Ejemplo: Creamos una colección nueva llamada nombres.

Comandos `insertMany()`

```
> db.nombres.insertMany([ { "nombre": "Juan Pablo", "edad": 28 }, { "nombre": "Sebastian Sanchez",  
"edad": 28 }, { "nombre": "Mario Gonzalez", "edad": 28 }, { "nombre": "David Penal", "edad": 28 }, {  
"nombre": "Esteban Sanchez", "edad": 28 }, { "nombre": "Christian Gomez", "edad": 22 }, {  
"nombre": "Angel Maria", "edad": 23 }, { "nombre": "Juan Carlos", "edad": 24 }, { "nombre": "Jose  
Maria", "edad": 30 }, { "nombre": "Robert Junior", "edad": 35 }, { "nombre": "David Bowie", "edad":  
60 }, { "nombre": "Felipe Gonzalez", "edad": 50 }, { "nombre": "Michael Jackson", "edad": 41 }, {  
"nombre": "Raul Gonzalez", "edad": 38 }, { "nombre": "Sergio Busquets", "edad": 35 }, { "nombre":  
"Lionel Messi", "edad": 33 }, { "nombre": "Cristiano Ronaldo", "edad": 34 }, { "nombre": "Enrique  
Iglesias", "edad": 37 }, { "nombre": "David Beckham", "edad": 41 }, { "nombre": "Silvester Stalone",  
"edad": 50 }, { "nombre": "Ricky Martin", "edad": 35 }])
```

```
> db.nombres.find().pretty()
```

No devuelve todas las filas, hay que escribir la palabra "it".

OPERACIONES CRUD



Ejemplo:

Comandos **find()**

Find devuelve un objeto de cursor. Vamos a forzarlo a un arreglo.

➤ `db.nombres.find().toArray()`

Otra forma es utilizando el método for each:

➤ `db.nombres.find().forEach((nombres)=>{printjson(nombres)})`

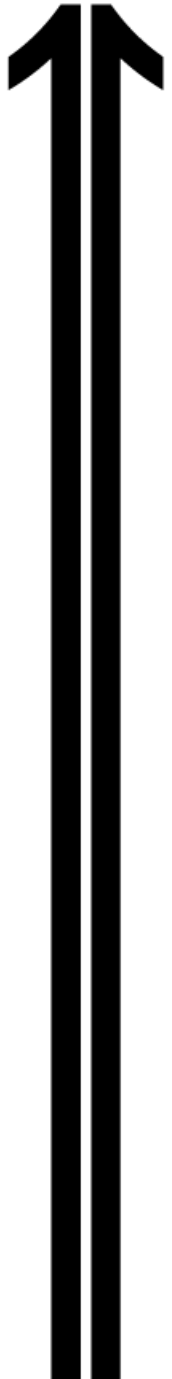
Podemos utilizar una metodología que se llama Proyección:

➤ `db.nombres.find({}, {nombre:1})`

La primera {} indica que devolverá todos los documentos, los demás campos tendrán un 1 o un 0 para visualizarlo.

➤ `db.nombres.find({}, {nombre:1 ,_id:0})`

El valor 0 indica que no mostrará ese campo, el caso del ID es especial porque siempre se mostrará.



Business intelligence, Fases de un
proyecto BI.

BUSINESS INTELLIGENCE (BI)



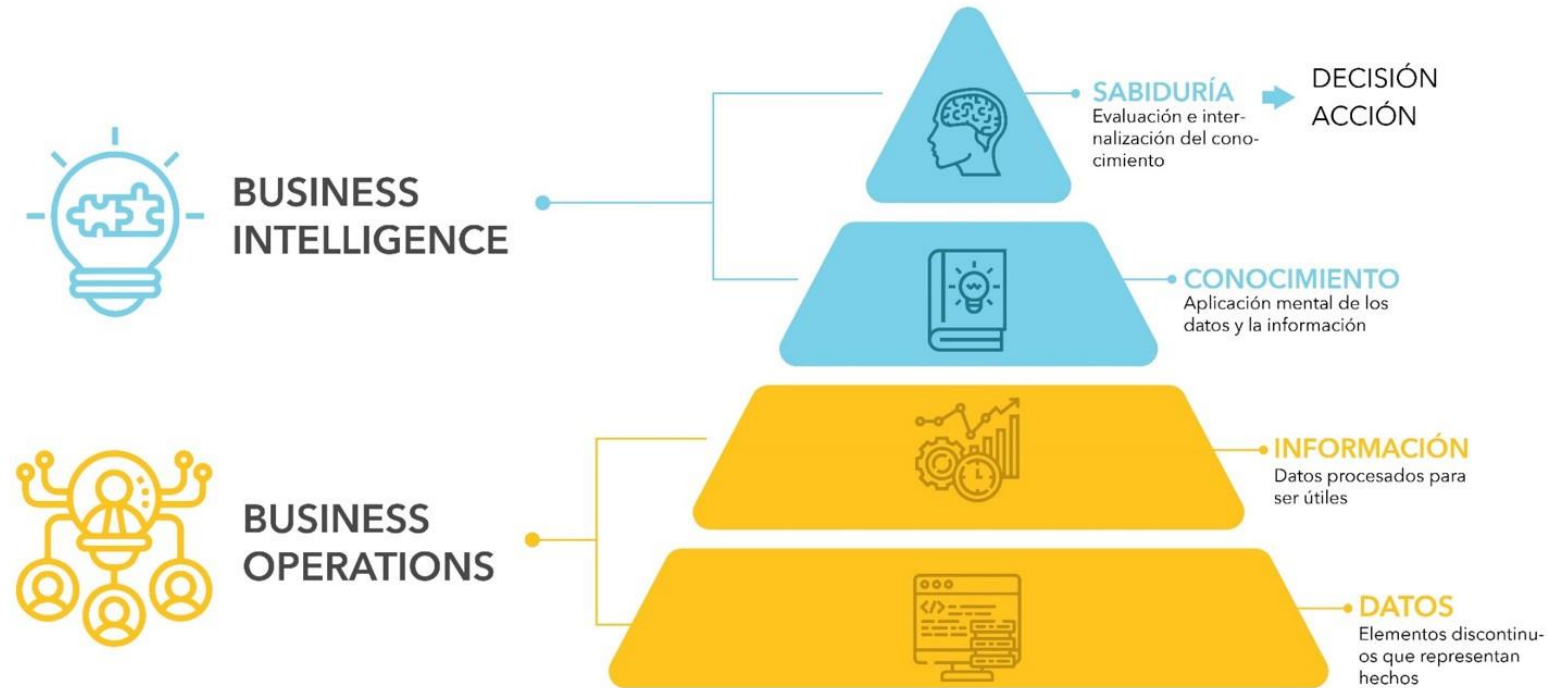
La Inteligencia de Negocios o Business Intelligence (BI) se puede definir como el proceso de **extraer datos** de cualquier fuente: archivos o base de datos para transformarlos en **información**, de tal manera que esta apoye a la **toma de decisiones** de las empresas. BI provee la información que requiere el usuario de negocio en la manera como lo desea y en el momento que lo necesite, mejorando enormemente la efectividad para la toma de decisiones, revelando tendencias de negocio no perceptibles fácilmente.

Es el conjunto de estrategias y herramientas orientadas a acceder, agrupar y guardar datos en una empresa, de forma tal que contribuyan a mejorar la performance y decisiones

BUSINESS INTELLIGENCE (BI)



La pirámide de la información



Fuente: Hey, J.: The Data, Information, Knowledge, Wisdom Chaim: The Metaphorical Link

BUSINESS INTELLIGENCE (BI)



Niveles de Información

En el nivel inferior: (Gestores Operativos) Tenemos la gestión pura y dura diaria, de tipo operativa, transaccional en donde se dispone de información muy estructurada , y en donde las decisiones se toman dentro de reglas y protocolos, buscando el seguimiento de objetivos e indicadores de orden operativo.

En el nivel Intermedio: (Mandos medios) La información de la que se dispone está semis-estructurada, entran en juego los planes Operativos previsiones y presupuestos, hay ciertos protocolos y procedimientos de trabajo y se dispone de algunos indicadores intermedios.

En el Nivel Superior: (Alta Gerencia) Se dispone de información des estructurada , hay planes estratégicos , las decisiones se toman con Base en servicios y necesidades de las personas y se dispone de indicadores clave.



“Es transformar datos en conocimiento”

La siguiente figura plasma de forma gráfica las diferencias entre datos, información y conocimiento:



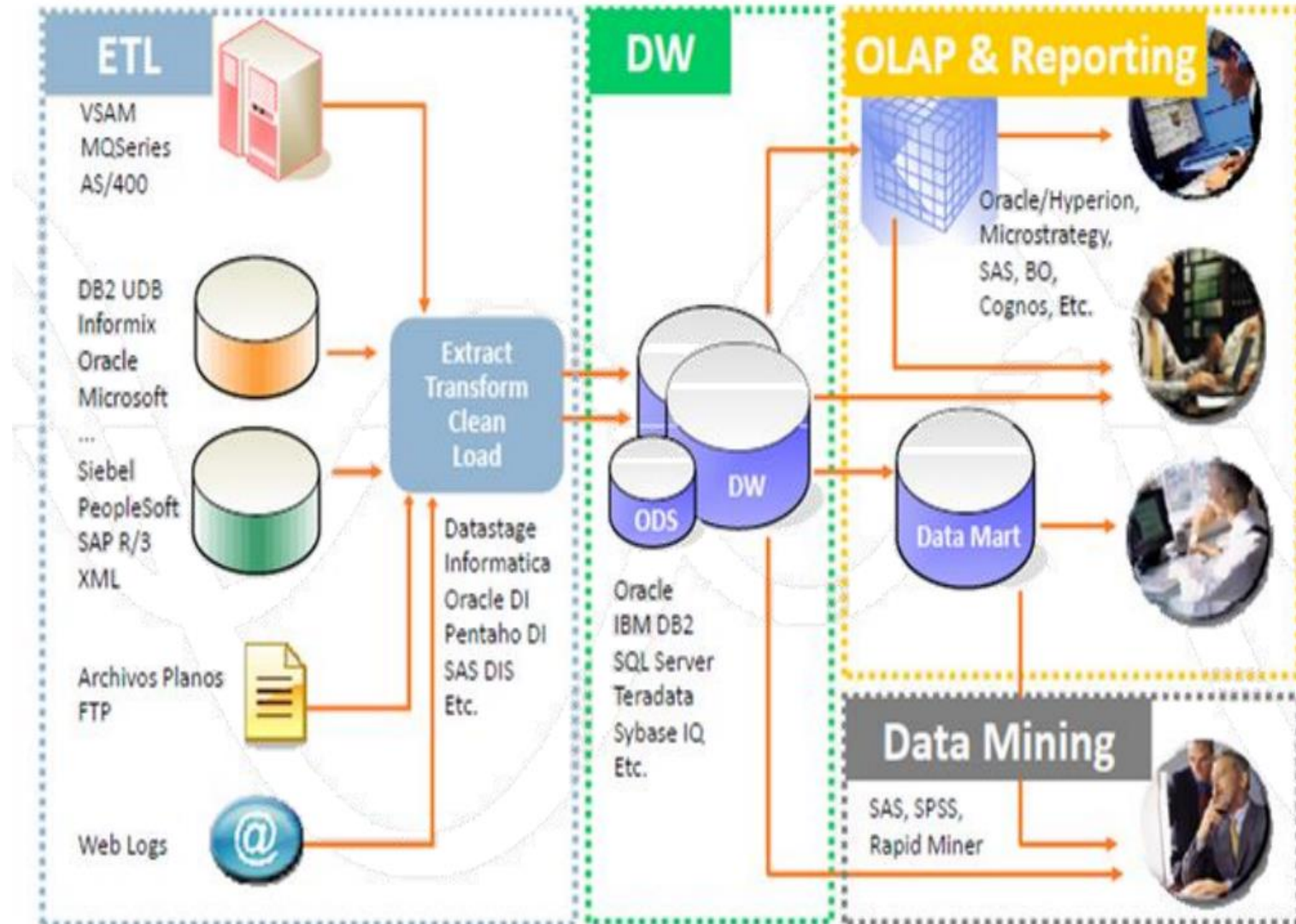
BUSINESS INTELLIGENCE (BI)



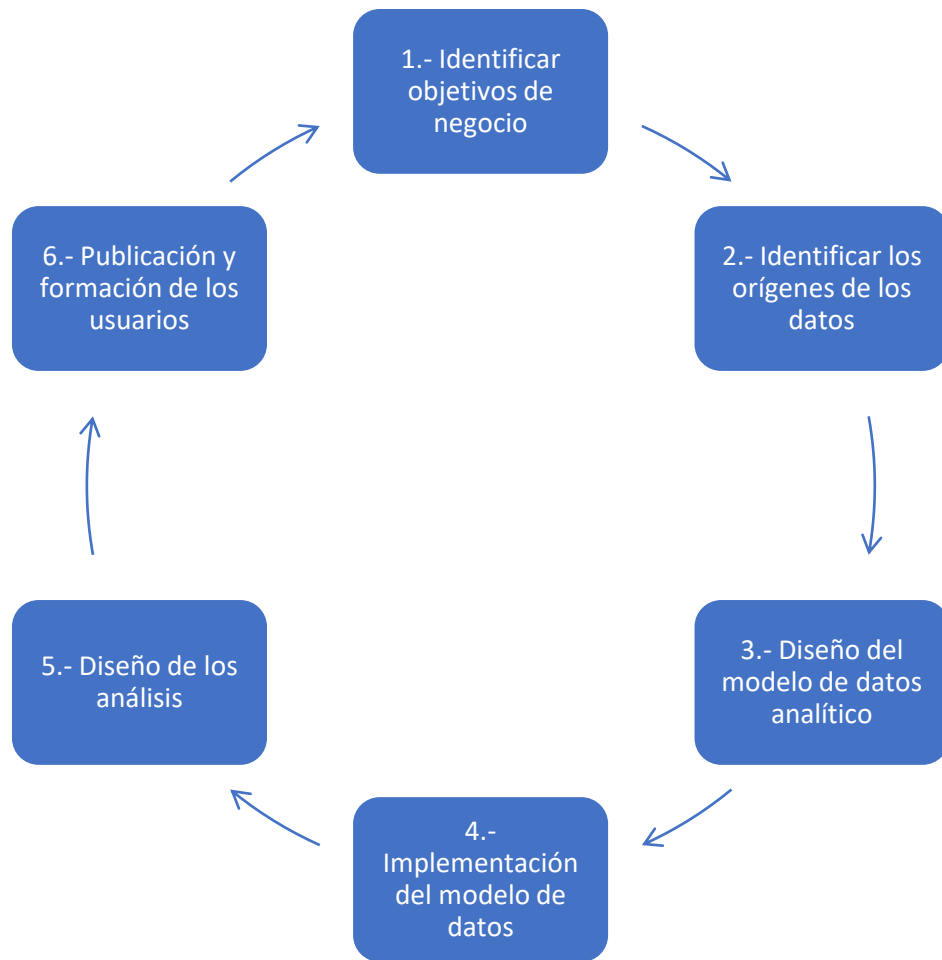
BUSINESS INTELLIGENCE (BI)



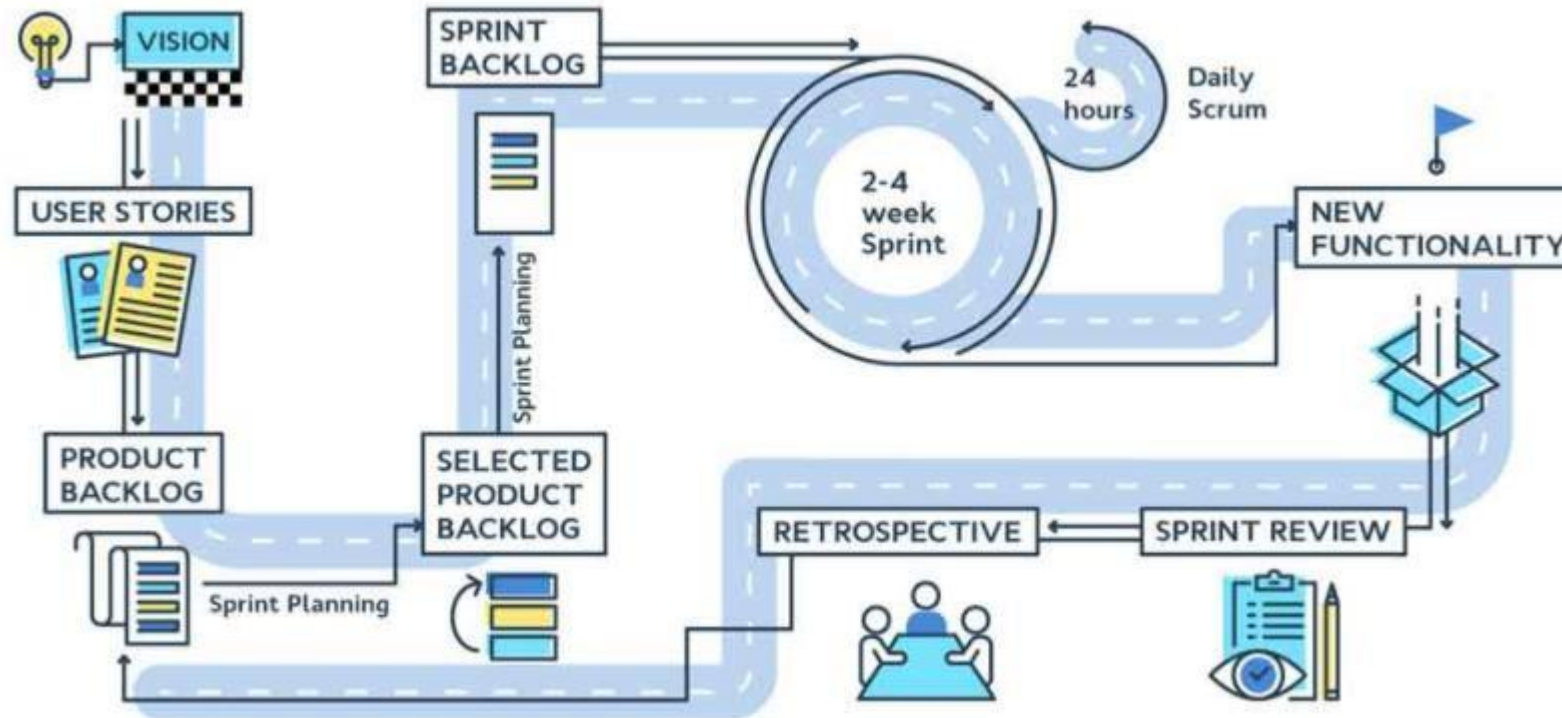
ARQUITECTURA DE BI



BUSINESS INTELLIGENCE (BI)



BUSINESS INTELLIGENCE (BI)





¿Preguntas o comentarios?



CONCLUSIONES

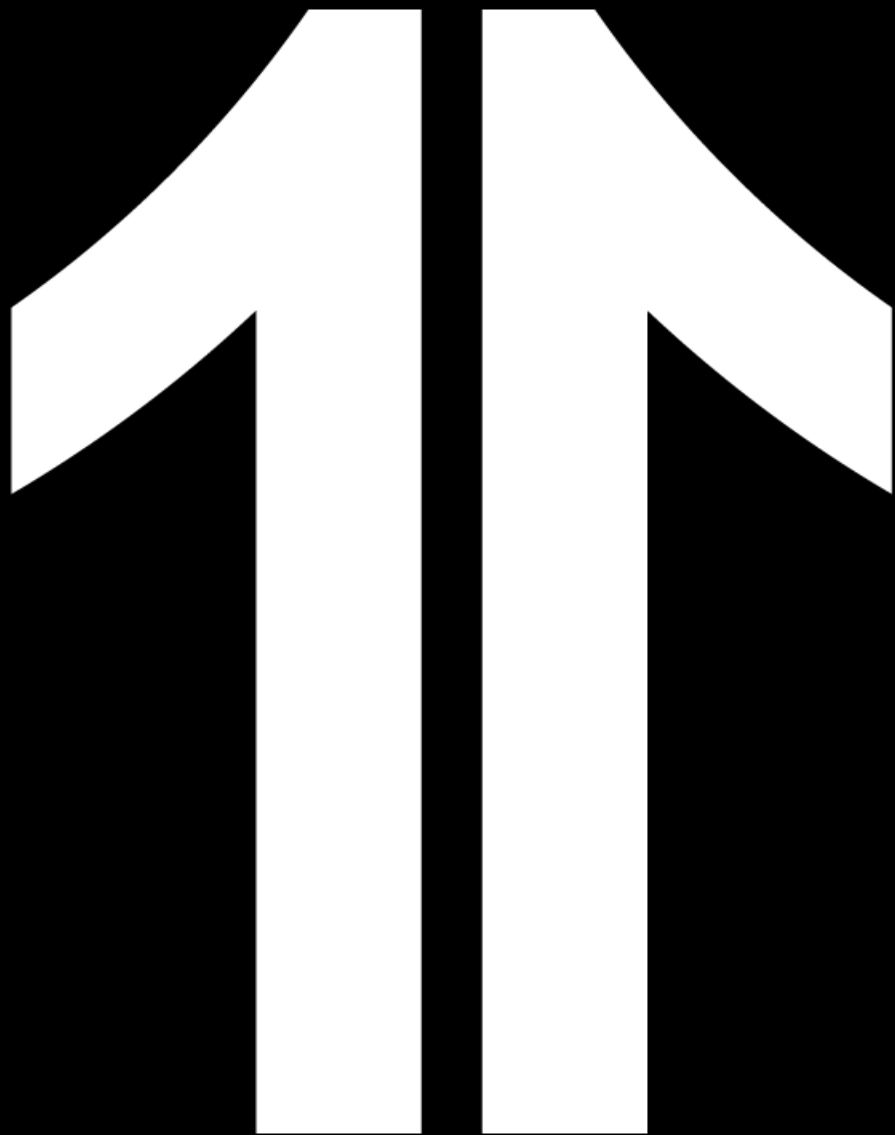
- Las bases de datos NoSQL son las más apropiadas para proyectos o empresas que tienen grandes cantidades de datos para almacenar. Además, la estructura mínima que proporcionan las bases de datos NoSQL puede permitir un desarrollo rápido para proyectos que lo necesiten.
- Las bases de datos NoSQL pueden no ser ideales para almacenar datos altamente estructurados.
- Los costos iniciales de las bases de datos NoSQL pueden ser muy bajos o incluso gratuitos. Sin embargo, después de los costos iniciales, mantener una base de datos NoSQL puede ser más caro dependiendo de cómo esté alojada. Además, los expertos en NoSQL son menos comunes que los expertos en SQL, por lo que la dotación de personal para un sistema NoSQL puede resultar más costosa.
- La característica más importante de una base de datos NoSQL a considerar es el modelo de datos que utiliza. A diferencia de las bases de datos SQL, que utilizan un modelo relacional, las bases de datos NoSQL utilizan una variedad de modelos diferentes. El modelo de valor clave es el más escalable a costa de la complejidad, mientras que el modelo de documento está optimizado para su uso con lenguajes de programación orientados a objetos.
- Existen muchos modelos de datos, así que se puede considerar cuál es el mejor para nuestras necesidades.



REFERENCIAS BIBLIOGRÁFICAS



- Fundamentos de las bases de datos NoSQL (<https://www.mongodb.com/es/nosql-explained>)
- NoSQL Databases (<https://www.trustradius.com/nosql-databases>)
- Instalación onPremise en Windows de MongoDB Community Edition (<https://docs.mongodb.com/manual/tutorial/install-mongodb-on-windows/>)
- Tipo de Datos (<https://docs.mongodb.com/manual/reference/bson-types/>)



GRACIAS

